

ORF307

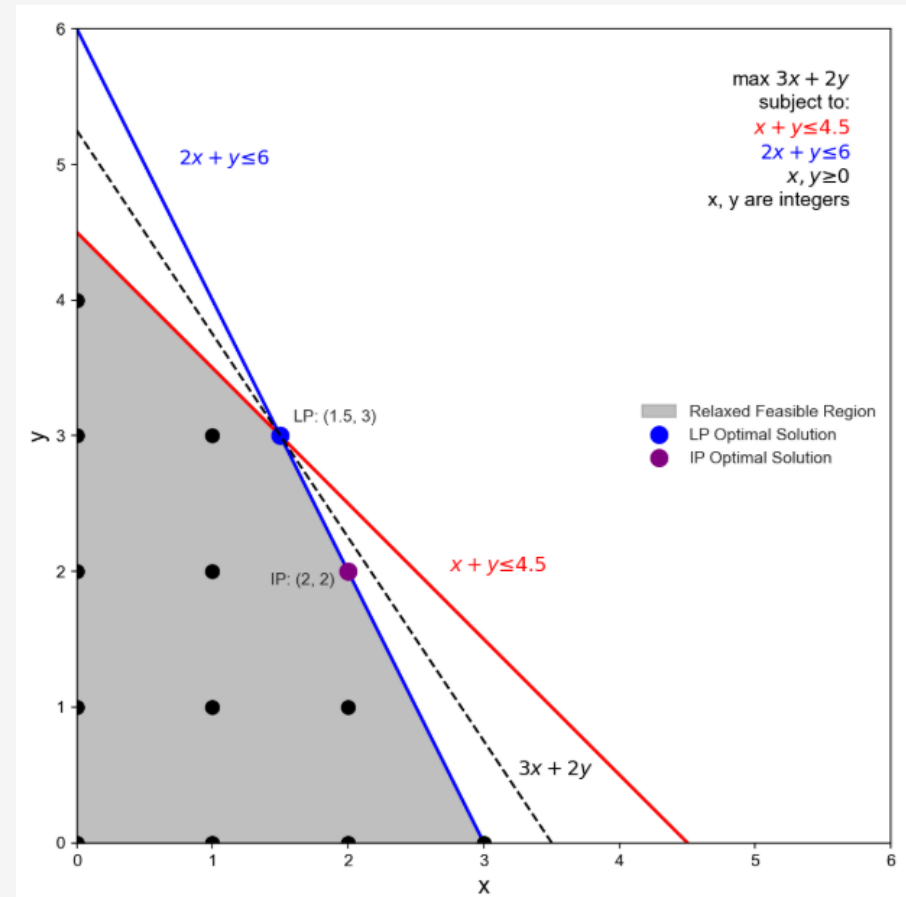
Integer Optimization Algorithms

Ioannis Akrotirianakis

Today's topics

- Branch and bound algorithm
- Branch and bound rules
- Examples
- Cardinality minimization

Check Canvas and EdDiscussion for details about the Final Exam!!

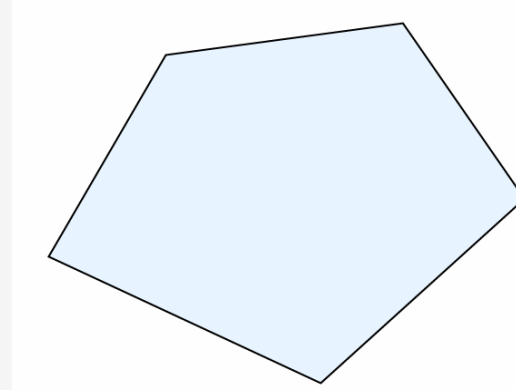
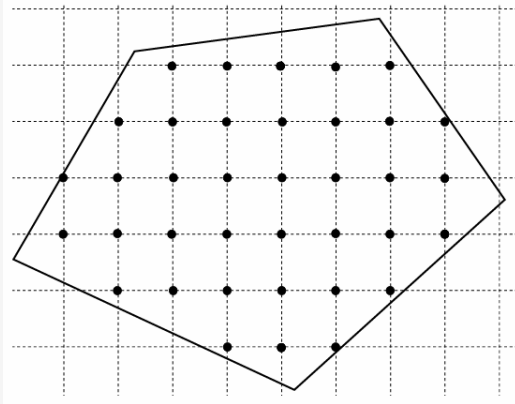


- **Recap**

Relaxations

Remove integrality constraints $\rightarrow$ allow integer variables to take real values

$$\begin{array}{ccc} \min c^T x & & \min c^T x \\ \text{s.t. } Ax \leq b & \longrightarrow & \text{s.t. } Ax \leq b \\ P_{ip} \longrightarrow x_i \in \mathbb{Z}, i \in \{1, \dots, n\} & & \longleftarrow P_{rel} \end{array}$$



$P_{ip} \subset P_{rel} \rightarrow$ relaxations provide lower bounds to p_{ip}^*

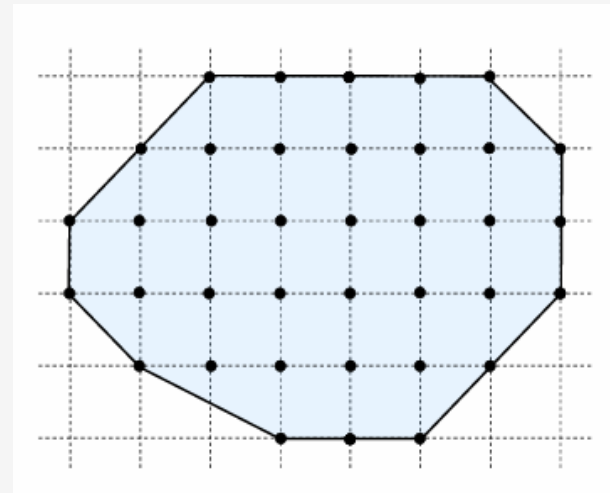
$$p_{ip}^* \leq p_{rel}^*$$

Ideal formulations

A formulation is called **ideal formulation** if solving its relaxation
gives an integer feasible solution

$$\begin{array}{lll} \min c^T x & \longrightarrow & \min c^T x \\ \text{s.t. } Ax \leq b & & \text{s.t. } Ax \leq b \\ x_i \in \mathbb{Z}, i \in \{1, \dots, n\} & & \end{array} \longrightarrow \begin{array}{l} \text{Integer feasible} \\ x^* \end{array}$$

This happens when $\text{conv } P = \{Ax \leq b\}$



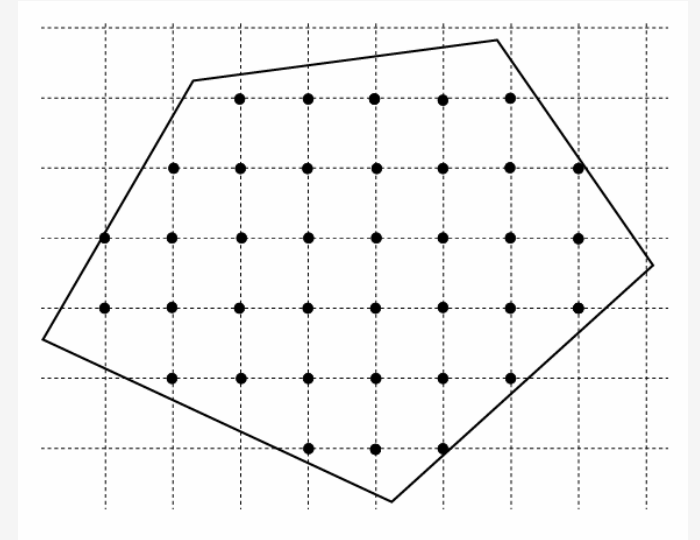
It is very hard to construct ideal formulations!

How do we solve integer optimization problems

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax \leq b \\ & x_i \in \mathbb{Z}^n, \quad i \in I \end{aligned}$$

Main idea: Refine the feasible set until the relaxation gives integer feasible solutions.

More details in the next lecture.



-
- **Branch and bound algorithm**

Example

How would you solve this MIP?

NOTE: only one variable (out of n) must be binary.

$$\begin{array}{ll}\min & c^T x \\ \text{s.t.} & Ax \leq b \\ & x_1 \in \{0,1\}\end{array}$$

$$c \in R^n, b \in R^m, A \in R^{m \times n}$$

You need to solve 2 LPs and consider the solution that gives the lowest objective value

How would you solve this MIP?
(How many LPs you need to solve?)

$$\begin{array}{ll}\min & c^T x \\ \text{s.t.} & Ax \leq b \\ & x_i \in \{0,1\}, i = 1, \dots, 10\end{array}$$

- Solve $2^{10} = 1024$ LPs
- Parallelize solutions
- Warm-start: similar problems

Exponential explosion of the LPs that need to be solved

Branch and bound works in a smart way and tries to avoid solving all possible LPs

Branch and bound algorithm

$$\min c^T x$$

$$\text{s.t. } x \in P_{ip}$$

Feasible region: $P_{ip} = \{x \mid Ax \leq b, x_i \in \mathbb{Z}, i \in I\}$

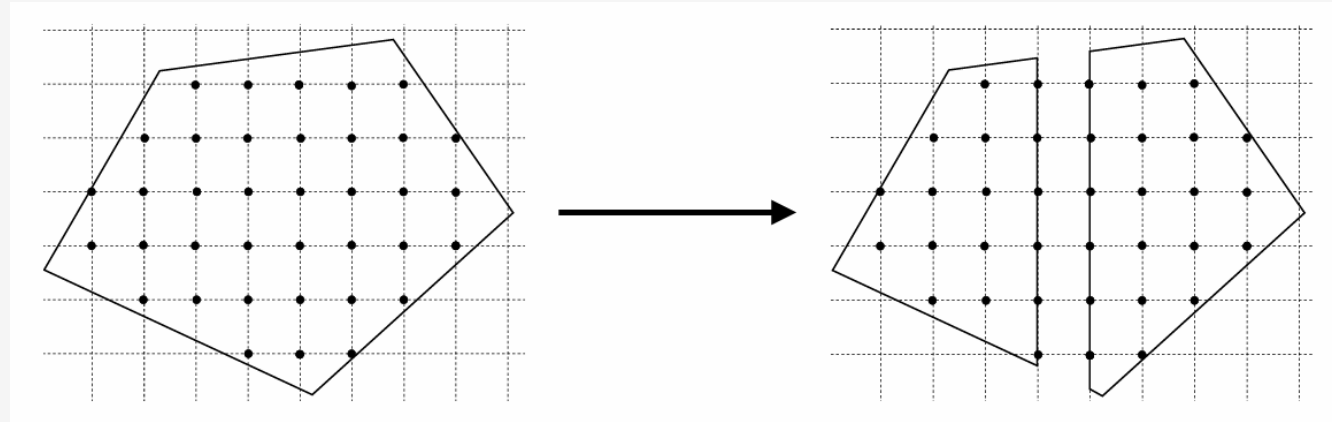
Divide and conquer:

- Partition P_{ip} in smaller sets $S^j \subset P_{ip}$
- Solve these subproblems:

$$\min c^T x$$

$$\text{s.t. } x \in S^j$$

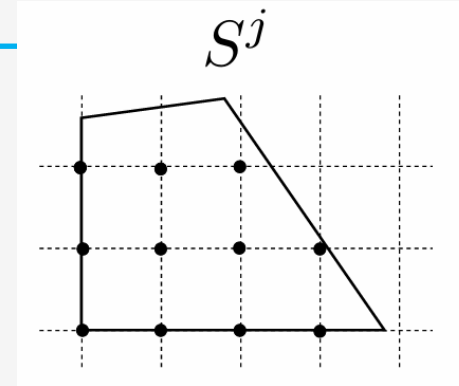
- Until optimal x^* is found



Two efficient subroutines

For every sub-region S^j define:

$$\Phi(S^j) := \min_{\text{s.t. } x \in S^j} c^T x := \min_{x \in S^j} c^T x$$



Lower and upper bounds:

Lower bound obtained
by solving LP relaxation



$$\Phi_{lb}(S^j) \leq \Phi(S^j) \leq \Phi_{up}(S^j)$$



Upper bound obtained by evaluating the objective at any feasible integer point

**Bounds are very important as they guide the algorithm
during the search for the integer optimal solutions x^***

Branch and bound algorithm

$$\begin{array}{ll} \min & c^T x \\ \text{s.t.} & x \in P_{ip} \end{array} \quad P_{ip} = \{x \mid Ax \leq b, \quad x_i \in Z, i \in I\}$$

Iterations

1. **Branch:** create/refine the partition of P_{ip} and get S^j

2. **Bound:**

- Compute **lower and upper bounds**

$$LB_j = \Phi_{lb}(S^j), \quad UB_j = \Phi_{ub}(S^j), \forall j$$

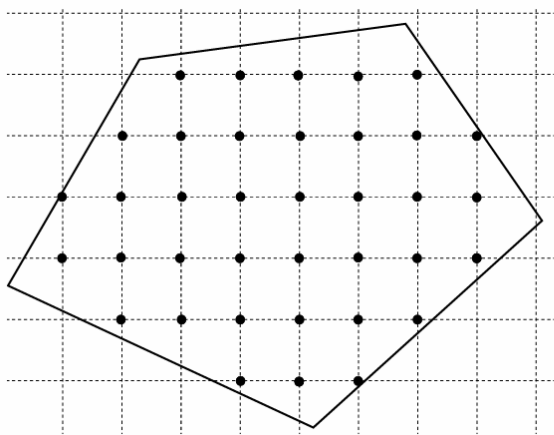
- **Update global bounds** on $c^T x^*$

$$LB = \min_j \{L_j\}, \quad UB = \min_j \{U_j\}$$

3. **If** $UB - LB \leq \epsilon$, **BREAK**

Branch and bound – Example in 2D

P_{ip}



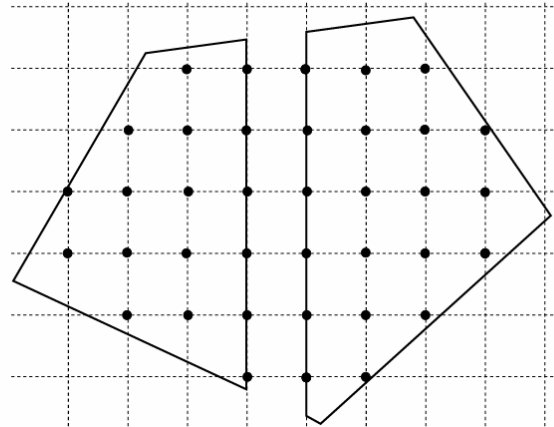
$$UB = 8.5$$

$$LB = 2$$



S^1

S^2



$$UB_1 = 8.5$$

$$LB_1 = 6$$

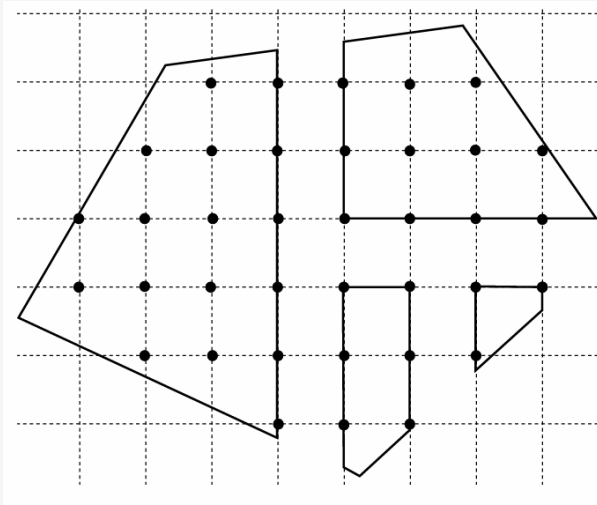
$$UB_2 = 4.2$$

$$LB_2 = 2$$

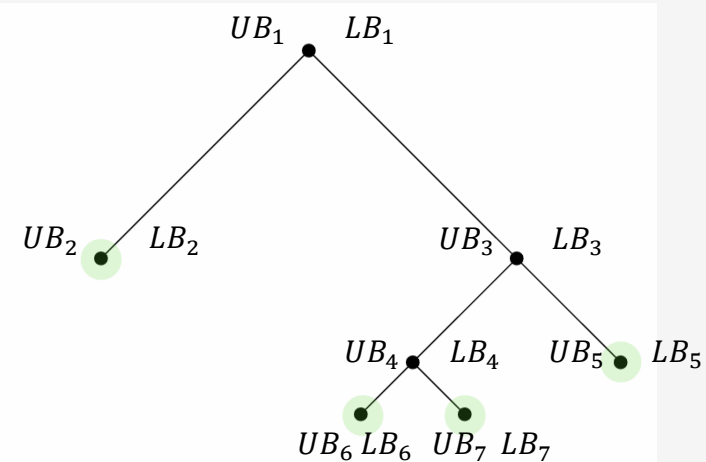
What does it say about
the global bounds L, U ?

Partition as a binary tree

Partitions



Binary tree



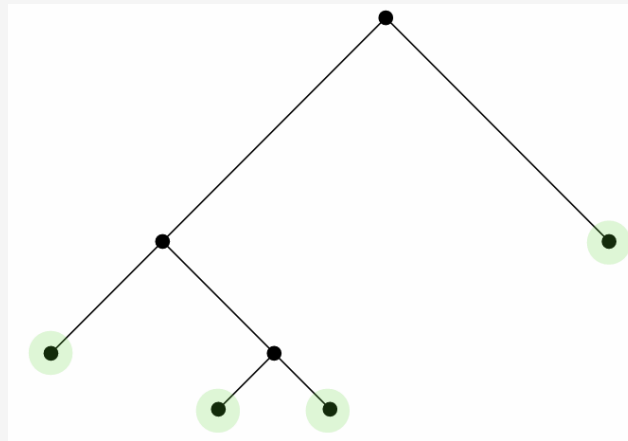
The solution process dynamically generates a binary tree

Children are derived by partitioning their parents into two subproblems

Certifying optimality

Certifies optimality $\longrightarrow$ $LB \leq c^T x^* \leq UB$ $\longleftarrow$ **Returns feasible point (incumbent)**

Partitions = Leaves



**Optimality certificate in
integer optimization**

Partition S^j
Bounds $(L_j, U_j), \forall j$

**Optimality certificate in
LP relaxations**

Dual variables and cost

-
- **Branch and bound rules**

Initializations

Original IP problem

$$\begin{array}{ll} \min & c^T x \\ \text{s.t.} & x \in P_{ip} \end{array}$$

LP relaxation

$$\begin{array}{ll} \min & c^T x \\ \text{s.t.} & x \in P_{lp} \end{array}$$

- **Incumbent** solution: $\hat{x}$ (just reserve space in the memory of your computer for future use) $\rightarrow \hat{x} = NULL$
 - The incumbent solution is used to carry the best integer feasible solution found so far in the B&B tree
- Initialize **global** Upper and Lower Bounds: $LB = -\infty, \quad UB = +\infty$
 - The global LB is updated by the optimal solution of an LP relaxation
 - The global UB is update by an objective function value of the incumbent solution

Partitioning

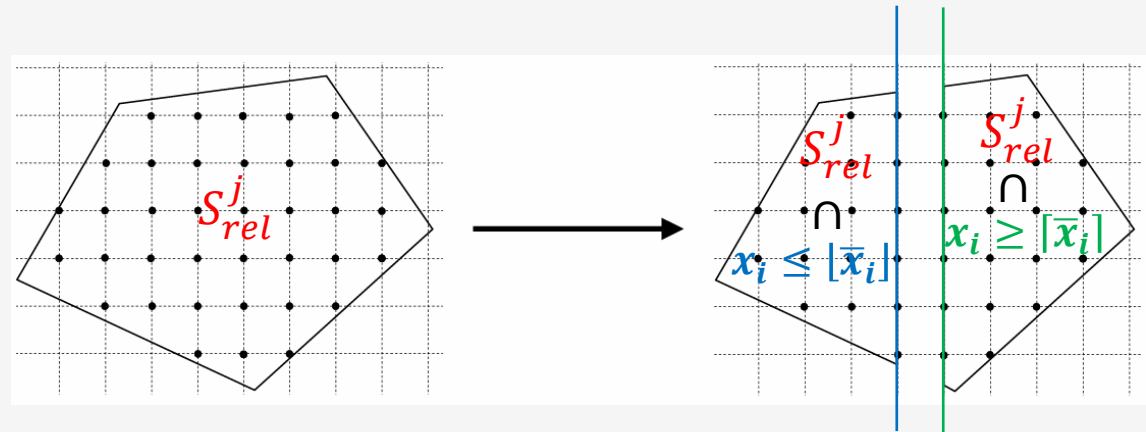
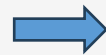
Pick one subproblem and solve its relaxation

$$\bar{x} \leftarrow \begin{array}{ll} \min & c^T x \\ \text{s.t.} & x \in S_{rel}^j \end{array}$$



Two possible cases

- If $\bar{x} \notin \mathbf{Z}^n$, there is $i \in I$ such that $\bar{x}_i \notin \mathbf{Z}$. We partition on that $\bar{x}_i$



$$\begin{array}{ll} \min & c^T x \\ \text{s.t.} & x \in S_{rel}^j \\ & x_i \leq \lfloor \bar{x}_i \rfloor \end{array}$$

$$\begin{array}{ll} \min & c^T x \\ \text{s.t.} & x \in S_{rel}^j \\ & x_i \geq \lceil \bar{x}_i \rceil \end{array}$$

- If $\bar{x} \in \mathbf{Z}^n$, then it is the optimal solution in S_{rel}^j



Update **incumbent** solution: $\hat{x} = \bar{x}$

No partition

Branching rules

Branching decisions

- Which subregion S^j to split?
- Which fractional variable $\bar{x}_i$ to split?

Goal:

- Get tight global bounds as quickly as possible
- They can dramatically reduce the computational time

Example heuristics (best-bound search)

- **Optimism**: split S^j with the lowest lower bound L_j
- **Greed**: split most fractional $\bar{x}_i$

Pruning

Key performance component

$$LB = \min_j LB_j \leq c^T x^* \leq \min_j UB_j = UB$$

Global LB **Global UB**

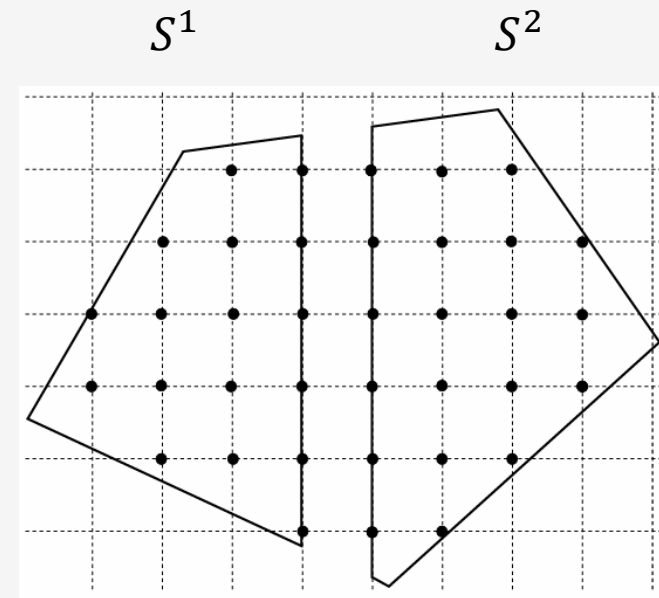
Subproblem S^j is **active** if $LB_j \leq \min_j UB_j$

Otherwise, it is **inactive** ($x^* \notin S^j$) and we can **prune** it

Questions:

What is S^1 ? Active or Inactive?

What is S^2 ? Active or Inactive?



$$\begin{array}{ll} UB_1 = 10 & UB_2 = 4.2 \\ LB_1 = 6 & LB_2 = 1.5 \end{array}$$

$$\begin{array}{l} UB = UB_2 \\ LB = LB_2 \end{array}$$

Bounding rules

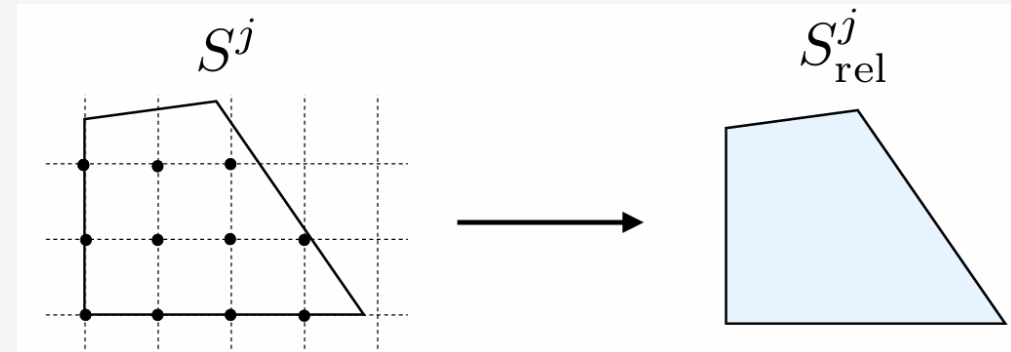
Lower bounds → Solve LP relaxation

$$\begin{aligned} \bar{x} \leftarrow \min c^T x \\ \text{s.t. } x \in S_{rel}^j \end{aligned}$$

- $LB_j = c^T \bar{x}$ valid lower bound to $c^T x^*$
- $LB_j = +\infty$, **if it is infeasible**

Upper bound

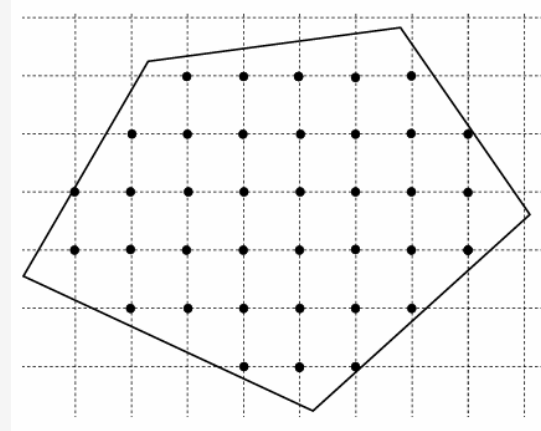
- $[\bar{x}] \leftarrow \text{Round } \bar{x}$ (LP relaxation solution)
- $UB_j = c^T [\bar{x}]$, if $[\bar{x}] \in S^j$
- $UB_j = \infty$, if $[\bar{x}] \notin S^j$ ($[\bar{x}]$ not feasible)



Branch and bound convergence

Binary Optimization problem

$$\begin{array}{ll}\min & c^T x \\ \text{s.t.} & Ax \leq b \\ & x \in \{0,1\}^n\end{array}$$



Branch and bound

- Worst-case: we may end up partitioning and solving **all 2^n subproblems**
- In practice we solve much fewer subproblems are solved

Brute force:

- Solve all 2^n subproblems, for all possible values $x \in \{0,1\}^n$
- It **blows up** for $n \geq 20$

Practical consideration

The solutions of subproblems are independent of each other

We can solve them **in parallel on multiple cores** of computing nodes.

Subproblems can be very similar

This is because **we only add just one constraint** ($x \leq \lfloor \bar{x}_i \rfloor$ or $x \geq \lceil \bar{x}_i \rceil$) at a time to the original feasible region ($Ax \leq b$).

We can warm-up the optimization algorithm we use.

Question: Which algorithm you would use to solve the subproblems?

The **dual Simplex** because $(\bar{y}, 0)$ is **still feasible** of the children subproblems

Note: $\bar{x}$ is optimal of the parent node and $\bar{y}$ is the optimal of the dual of the parent problem

-
- **Small examples**

Branch and bound example

$$\min -2x_1 - 3x_2$$

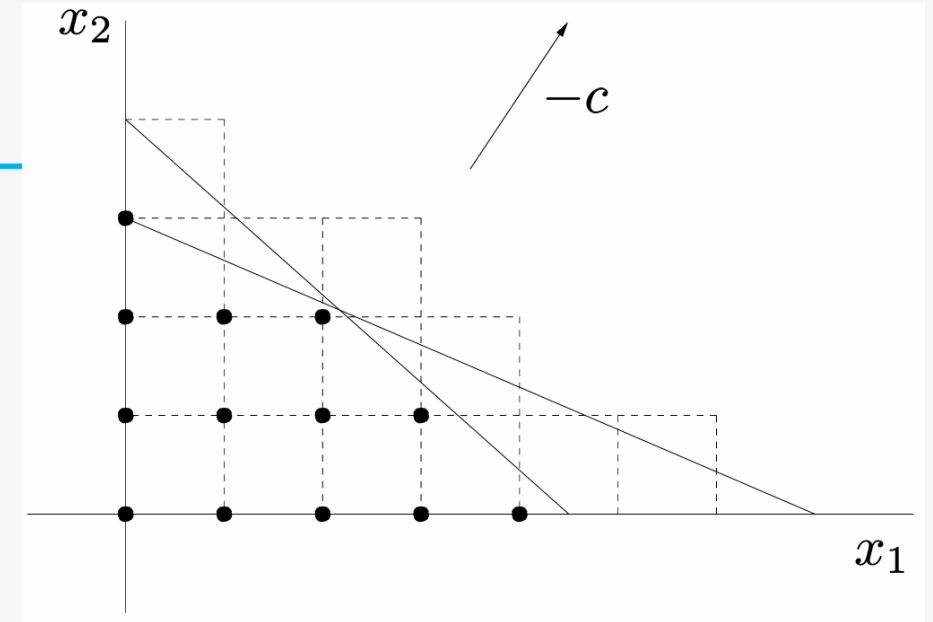
$$\text{s.t. } (2/9)x_1 + (1/4)x_2 \leq 1$$

$$(1/7)x_1 + (1/3)x_2 \leq 1$$

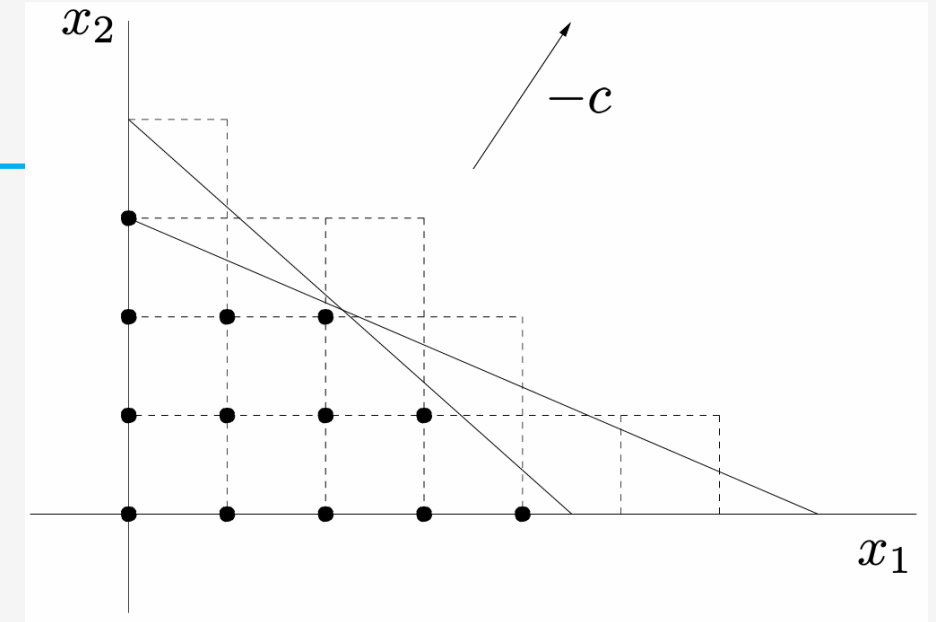
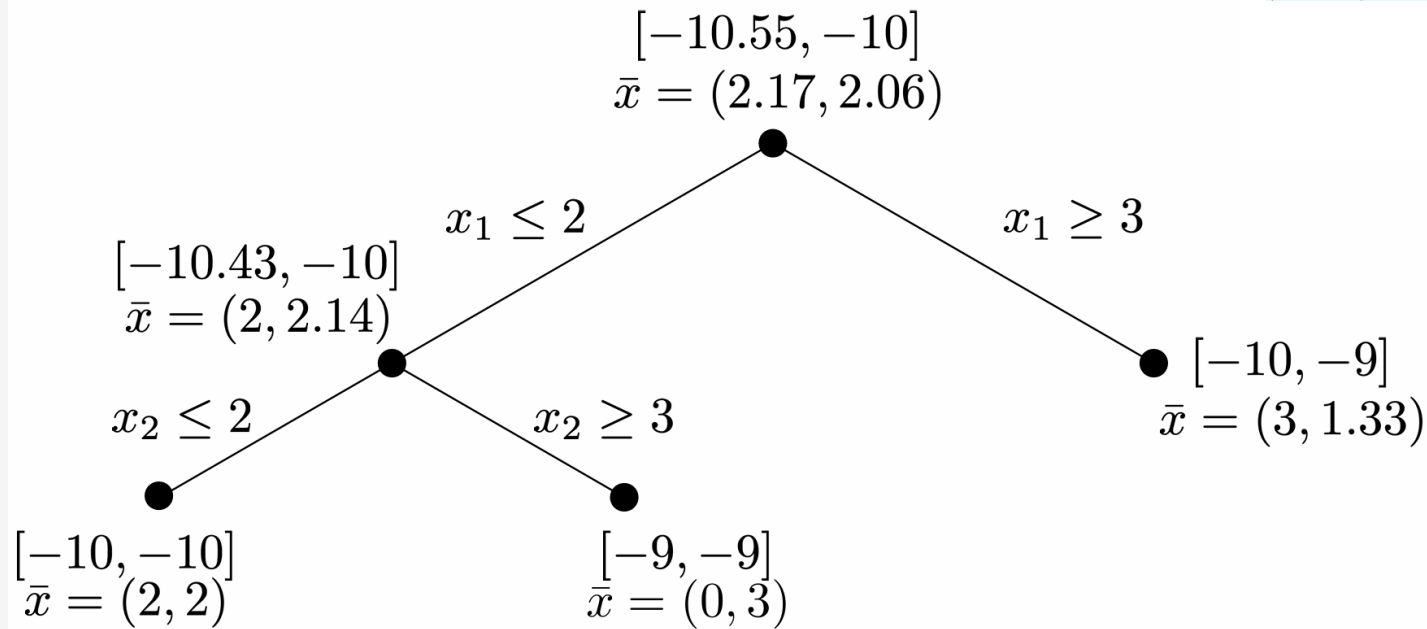
$$x_1, x_2 \geq 0$$

$$x_1, x_2 \in \mathbb{Z}$$

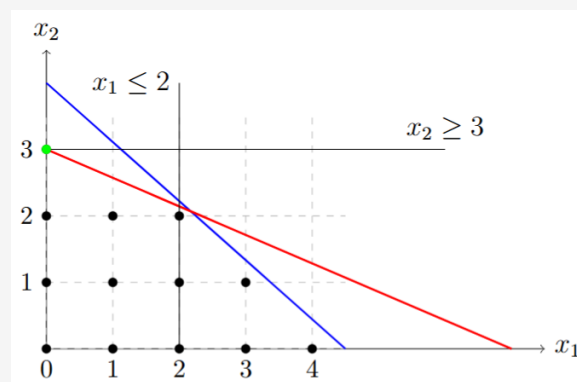
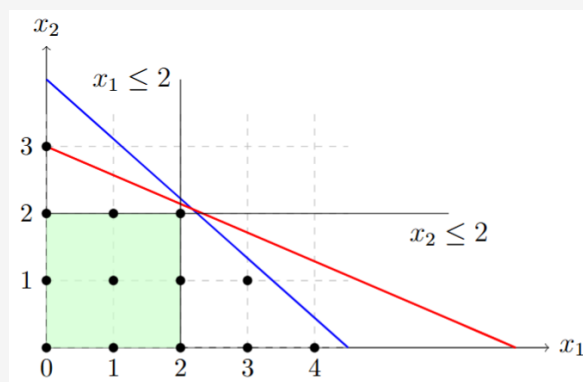
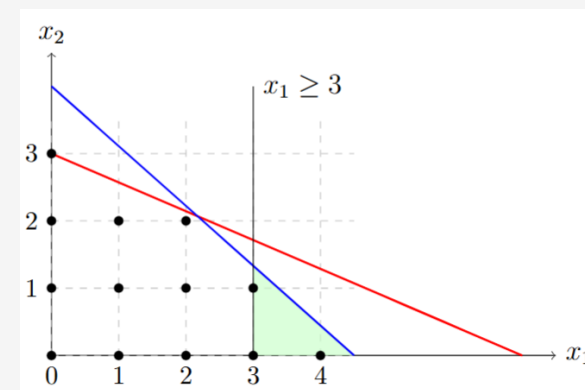
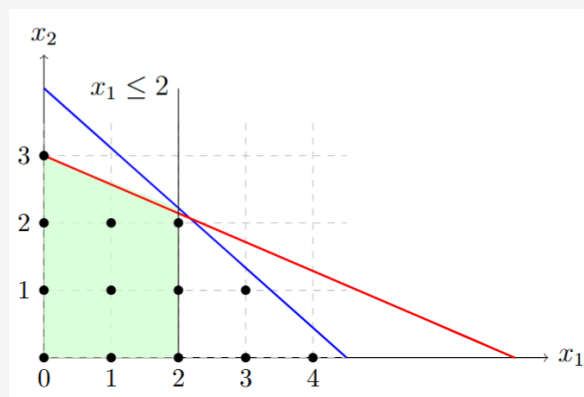
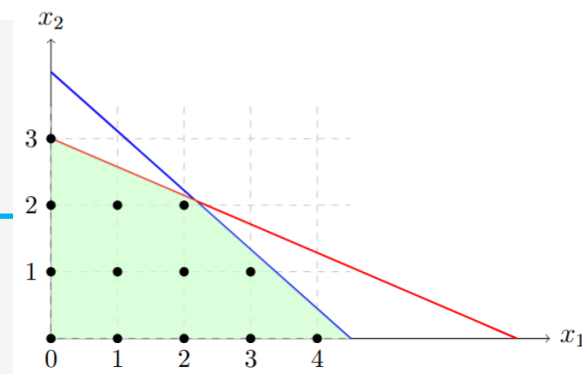
Optimal solution: $x^* = (2,2)^T$



Branch and bound example

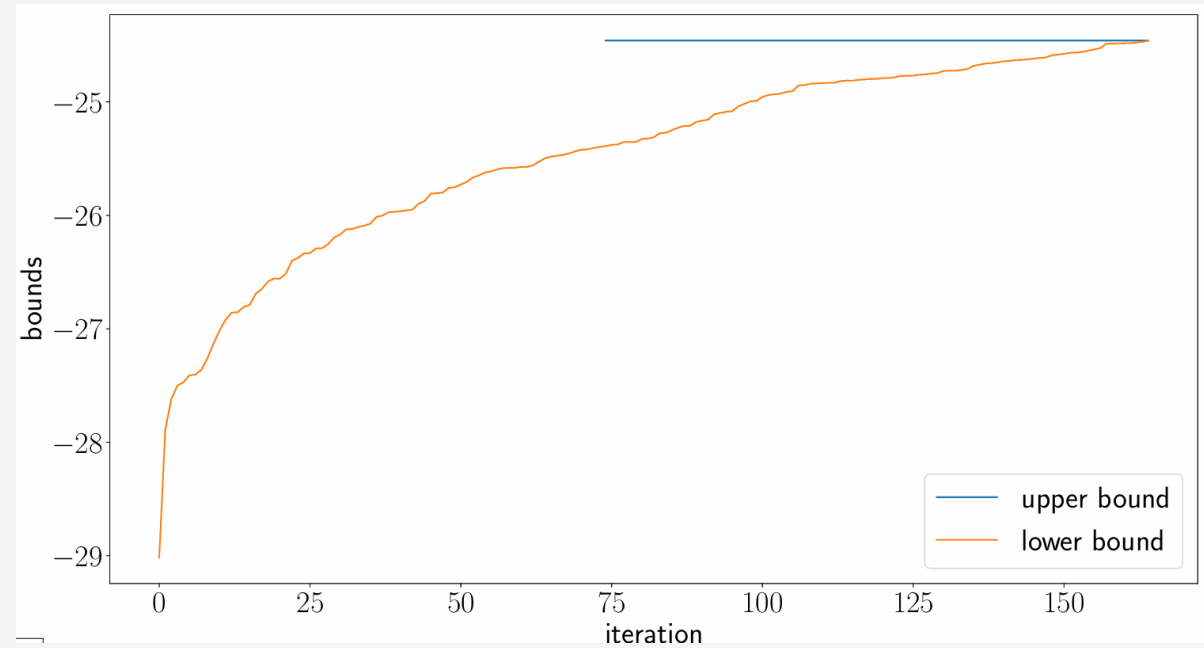
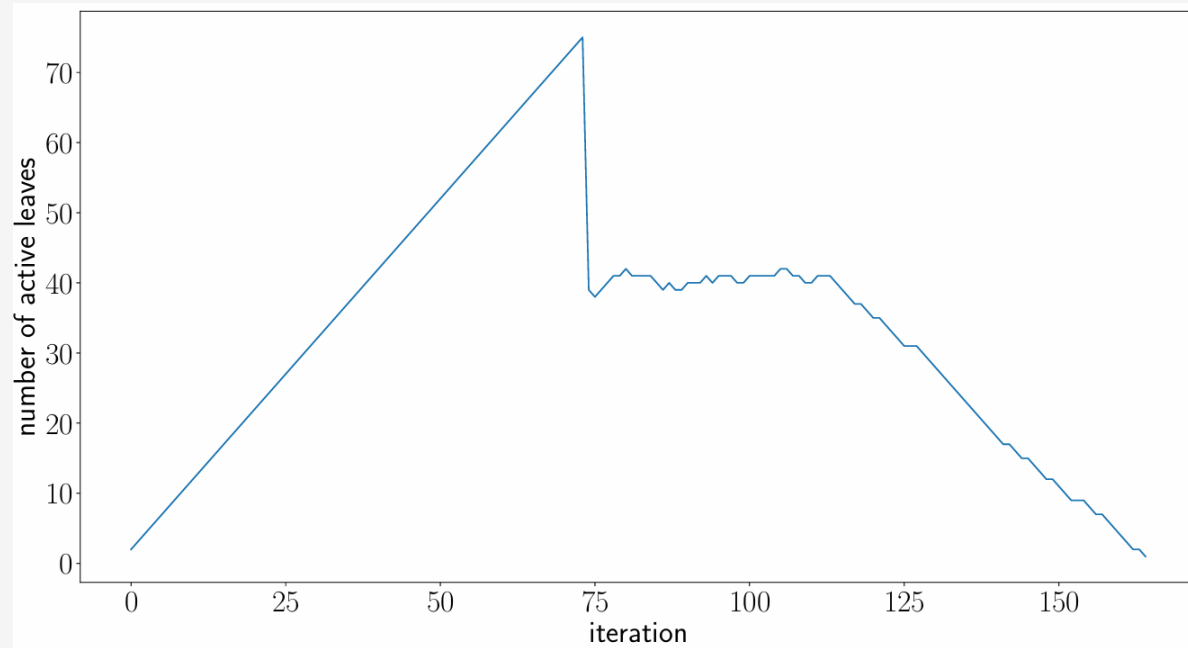


See **Precept 11** for detailed computations and the partition of the feasible region



Larger example

$$\begin{array}{ll} \min & c^T x \\ \text{s.t.} & Ax \leq b \\ & x \in \mathbb{Z}^n \end{array} \quad \begin{array}{l} m = 20 \\ n = 10 \end{array}$$



Cardinality optimization

Minimum cardinality example

Find **the sparsest** x satisfying a set of linear inequalities

$$\begin{array}{ll}\min & \text{card}(x) \\ \text{s.t.} & Ax \leq b\end{array}$$

Equivalent Mixed-Boolean LP

$$\begin{array}{ll}\min & \mathbf{1}^T z \\ \text{s.t.} & l_i z_i \leq x_i \leq u_i z_i, \quad i = 1, \dots, n \\ & Ax \leq b \\ & z \in \{0,1\}^n\end{array}$$

Big-M formulation

- l_i and u_i are lower/upper bounds on x_i (you could set $l_i = -M$ and $u_i = M \rightarrow$ **inefficient**)
- The tightness of the l_i, u_i can **greatly affect the convergence** of BB algorithm

It can dramatically reduce the size of the B&B tree

Computing big-M constants

l_i is the optimal value of $\min x_i$
s.t. $Ax \leq b$

Computational cost:
Solution of $2n$ LPs

u_i is the optimal value of $\max x_i$
s.t. $Ax \leq b$

Remarks:

- If $l_i > 0$ or $u_i < 0$ then we can safely set: $z_i = 1$ **(Why? Exercise!)**
 - We cannot have $x_i = 0$
- This procedure, called "**bound tightening**", is very common in the pre-processing step of modern optimization solvers

$$\begin{aligned}
& \min \mathbf{1}^T z \\
& \text{s.t. } l_i z_i \leq x_i \leq u_i z_i, \quad i = 1, \dots, n \\
& \quad Ax \leq b \\
& \quad z \in \{0,1\}^n
\end{aligned}$$

If $u_i = l_i = M$ then

$$-Mz_i \leq x_i \leq Mz_i \quad \Rightarrow \quad (1/M)|x_i| \leq z_i$$

ℓ_1 -norm minimization

$$\begin{aligned}
& \min (1/M) \|x\|_1 \\
& \text{s.t. } Ax \leq b
\end{aligned}$$

The Relaxation of the Cardinality problem is an alternative version of ℓ_1 -norm minimization

- They both induce sparsity

Implementation details

Upper bound: $\text{card}(\bar{x})$ with $\bar{x}$ from the relaxation (ℓ_1 -norm induces sparsity)

Lower bound: we can replace LB with $\lceil LB \rceil$ since $\text{card}(\cdot)$ is integer valued function

Best-bound search: split node with lowest LB

Most ambivalent variable: the closest z_j to $1/2$

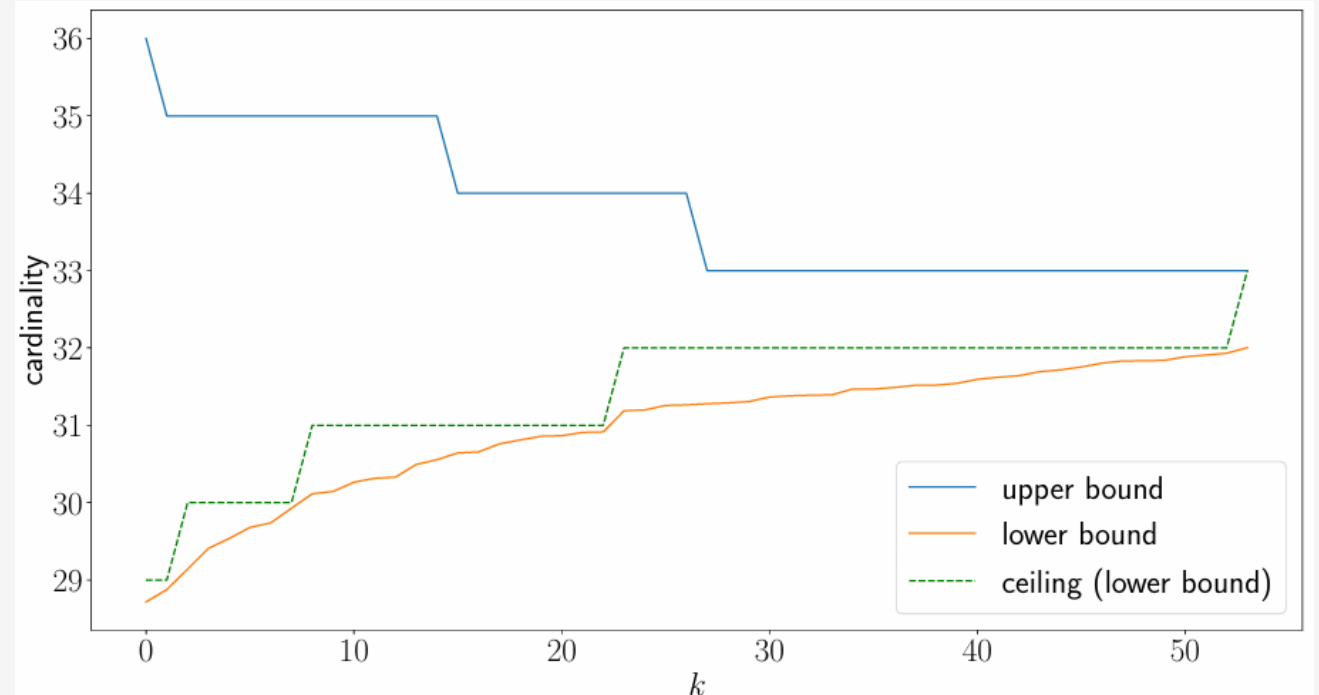
Small-scale example

Data

- 40 variables and 200 constraints
- $2^{40} \approx 1$ trillion combinations

Results

- Finds good solution very quickly
- Weighted ℓ_1 -norm heuristic works very well
- Terminates in 54 iterations



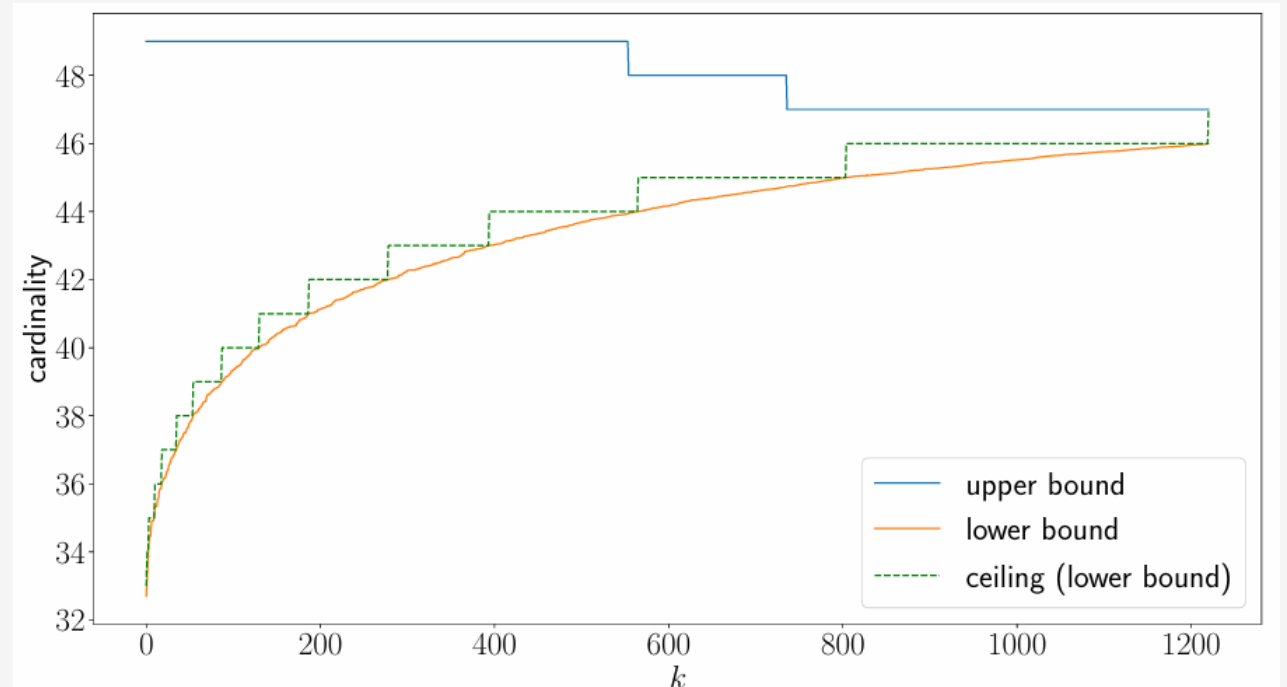
Medium-scale example

Data

- 60 variables and 200 constraints
- $2^{60} \approx 1.15 \times 10^{18}$ combinations

Results

- Finds good solution very quickly
- Weighted ℓ_1 -norm heuristic works very well
- Terminates in ≈ 1200 iterations



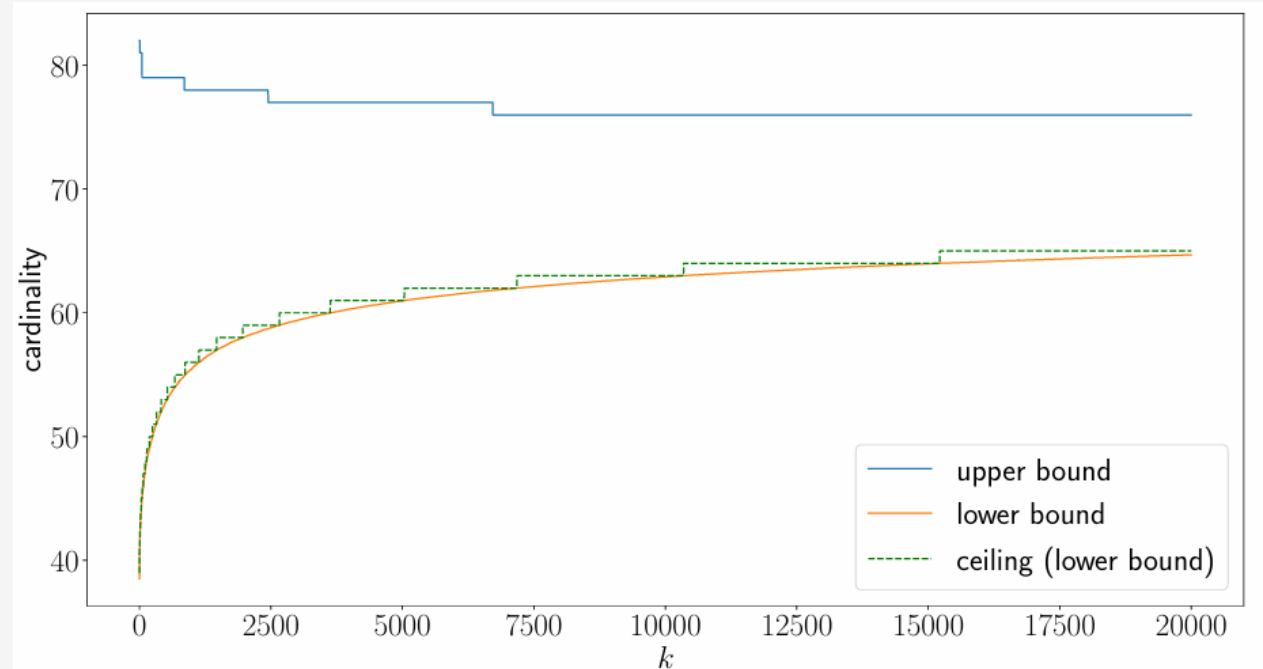
Large-scale example

Data

- 100 variables and 300 constraints
- $2^{100} \approx 1.26 \times 10^{30}$ combinations

Results

- Finds good solution very quickly
- 6 hours run with no termination
- Only optimality gap $U - L$ in the end



Large-scale example – with commercial solver *Gurobi*

Data

- 100 variables and 300 constraints
- $2^{100} \approx 1.26 \times 10^{30}$ combinations

Results

- Optimal cardinality: 72
- Uses much more sophisticated algorithm
- 1888 seconds (~31 minutes) run (slow!)

```
Gurobi Optimizer version 9.0.3 build v9.0.3rc0 (mac64)
Optimize a model with 500 rows, 200 columns and 30400 nonzeros
Variable types: 100 continuous, 100 integer (100 binary)
Coefficient statistics:
  Matrix range      [4e-05, 5e+00]
  Objective range   [1e+00, 1e+00]
  Bounds range     [1e+00, 1e+00]
  RHS range        [4e-03, 3e+01]
Presolve time: 0.05s
Presolved: 500 rows, 200 columns, 30400 nonzeros
Variable types: 100 continuous, 100 integer (100 binary)

Root relaxation: objective 2.933185e+01, 735 iterations, 0.18 seconds

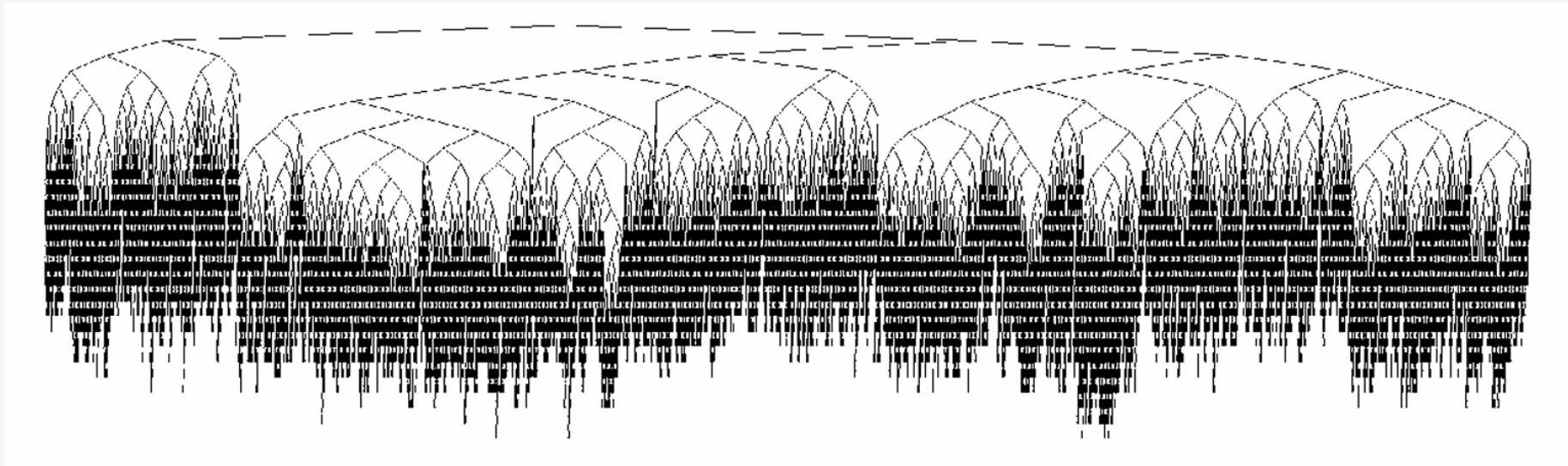
   Nodes      |   Current Node   |   Objective Bounds   |   Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap   | It/Node Time
-----
    0     0    29.33185    0   85      -       29.33185    -       -     0s
H    0     0      30.18570    0   83    85.000000    29.33185   65.5%    -     0s
    0     0      30.18570    0   83    85.000000    30.18570   64.5%    -     1s
H    0     0      31.35255    0   86    83.000000    30.18570   63.6%    -     1s
    0     0      31.81240    0   86    83.000000    31.35255   62.2%    -     2s
    0     2      31.81240    0   86    83.000000    31.81240   61.7%    -     3s
H   271    73      47.90892   36   47    82.000000    35.05009   57.3%   58.6     4s
   376   104      47.90892   36   47    82.000000    35.05009   57.3%   54.1     5s
...
2887987 13108      cutoff    88      72.00000    70.70801   1.79%   34.1 1880s
2897345 4880      cutoff    87      72.00000    70.86531   1.58%   34.1 1885s

Explored 2903463 nodes (98760290 simplex iterations) in 1888.42 seconds
Thread count was 16 (of 16 available processors)

Optimal solution found (tolerance 1.00e-04)
Best objective 7.2000000000000e+01, best bound 7.2000000000000e+01, gap 0.0000%
```

Tree size can grow dramatically

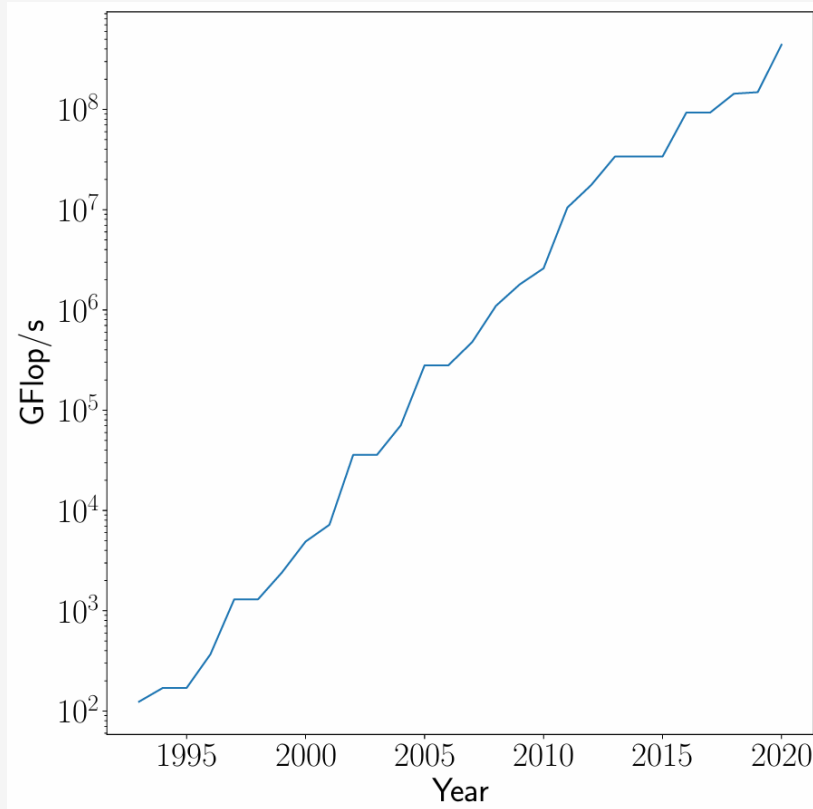
Example for 360 seconds on CPU and 10,000 nodes



10,000 nodes

Progress of mixed-integer optimization solvers

Top500 peak CPU power



**Hardware speedups
4 mln x**

**Hardware speedups
100,000 x**

**400 billion times
speedups**

400,000 years $\longrightarrow$ 30 seconds

Next lecture

- I hope you enjoyed the course and learned how to Optimize
- Optimization is very useful tool that I am sure you will use in other courses and in your careers.
- Thank you for keeping me company during Tuesdays and Thursdays!
- Good Luck on your final exams and hope to meet you again next semester!
